



ВТУ «Св. Св. Кирил и Методий»

Факултет «Математика и информатика»

РЕФЕРАТ

*Градивни шаблони за проектиране
(Creational design patterns)*

Разработил:

Николета Йончева Панайотова
Спец. “Информационни ситеми”
Фак. №: МПИ-10679р

Проверил:

гл. ас. д-р Ивайло Дончев

Велико Търново
2012г.

Съдържание

<u>Увод</u>	3
1. Създаващи шаблони (<i>Creational Design Pattern</i>)	4
1.1. Абстрактна фабрика (<i>Abstract Factory</i>)	5
1.2. Метод фабрика (<i>Factory Method</i>)	6
1.3. Строител (<i>Builder</i>)	10
1.4. Късна инициализация (<i>Lazy Initialization</i>)	12
1.5. Стингълтон (<i>Stingleton</i>)	12
1.6. Обектен пул (<i>Object Pool</i>)	14
1.7. Прототип (<i>Prototype</i>)	15
<u>Заклучение</u>	18
Използвана литература	18

УВОД

Понятието **“Шаблон за дизайн”** (*Design Pattern*) означава общо решение за често срещан проблем, което може да бъде използвано многократно. За първи път то се споменава през 1977г. от Кристофър Александър в книгата *“A Pattern Language: Towns, Buildings, Construction”*. От 1987г., благодарение на Кент Бек и Уард Кънингам тази идея започва да се възприема и в програмирането. Бурното ѝ развитие обаче започва през 1994г., когато напуска книгата *“Design Patterns: Elements of Reusable Object-Oriented Software”* от Ерих Гама, Ричард Хелм, Ралф Джонсън и Джон Влсидес. За кратко време тя става един от най-важните източници за знания в областта на ООП дизайн - теория и практика, както и една от най-продаваните и търсените книги за софтуерно инженерство - тенденция, запазила се до наши дни. Трябва да се отбележи, че описаните в нея шаблони не са разработка на авторите, а са резултат от изследване на много проекти и документиране на общите елементи в дизайна.

Софтуерните шаблоните за дизайн (*Software design pattern*) представляват концепция, предназначена за разрешаване на често срещани проблеми в обектно-ориентираното програмиране. Тази концепция предлага стандартни решения за архитектурни и концептуални проблеми в компютърното програмиране. Тук не става въпрос за конкретни алгоритми или част от програмен код. Шаблоните за дизайн са независими от програмния език. Те представляват архитектурни решения на вече познати и много често срещани проблеми в програмирането. Могат да бъдат реализирани на различни обектно-ориентирани езици като *C++*, *C#*, *Java*, *PHP* и други.

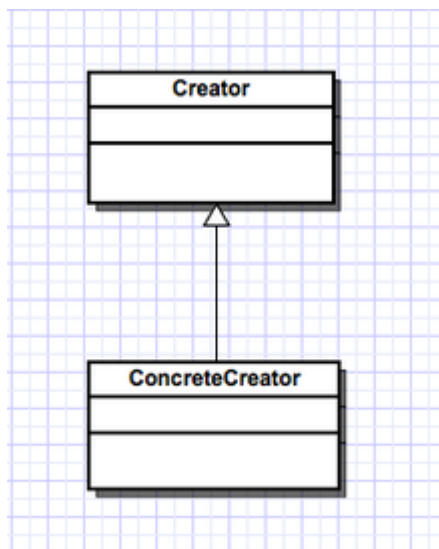
Шаблоните биват:

- Създаващи / Градивни (*Creational*);
- Структурни (*Structural*);
- Поведенчески (*Behavioral*);
- Архитектурни (*Concurrency*);

В настоящия реферат предстои да разгледаме създаващите шаблони.

1. Създаващи шаблони (*Creational design pattern*)

Това са шаблони, които се отнасят до създаването на нови обекти. Най често не се прибегва до директно викане на *new*, а се минава през специализиран клас/обект, който да върне желанния обект.



Фиг. 1.1 – Диаграма на клас от създаващ шаблон.

- **Creator:** декларира обектен интерфейс. Връща обект.
- **ConcreteCreator:** имплементира обектен интерфейс.

Основни създаващи шаблони:

- ♦ **Абстрактна фабрика (*Abstract Factory*)** - предоставя интерфейс за създаване на семейства от обекти, без да са посочени техните конкретни класове.
- ♦ **Метод Фабрика (*Factory Method*)** - дефинира интерфейс за създаване на обекти, но оставя на подкласовете да решат от кои класове да направят инстанции.
- ♦ **Строител (*Builder*)** - разделя създаването на сложен обект от неговото представяне, така че един и същи процес да може да създава обекти с различно представяне.
- ♦ **Късна инстанция (*Lazy Instantion*)** – представлява отлагане във времето на създаването на обект, изчисляването на стойност или на някакъв друг отнемащ ресурси процес, до момента в който не е нужен.
- ♦ **Сингълтон / Сек (*Singleton*)** - осигурява клас, който може да има само една единствена инстанция и предоставя глобален достъп до нея.
- ♦ **Обектен Пул (*Object Pool*)** - предотвратява скъпо заделяне или освобождаване на ресурс чрез рециклиране на обекти с кратък живот.
- ♦ **Прототип (*Prototype*)** - определя прототипна инстанция на някакъв вид обект и създава нови обекти чрез копиране на прототипа.

1.1. Абстрактна фабрика (*Abstract Factory*)

Фабриката е средство за създаване на обекти. Целта на този шаблон за дизайн е да изолира създаването на обектите от тяхното използване. Абстрактната фабрика капсулира група от методи Фабрика, имащи близко предназначение. Клиентският код създава конкретна имплементация на абстрактната фабрика, след това използва основния интерфейс, за да създава конкретни обекти. Клиентът не е задължен да знае коя от тези фабрики е създавала конкретния обект, защото той използва само основния интерфейс към създадените обекти. Този шаблон позволява замяната на конкретни класове, дори по време на изпълнение, без да е нужна промяна на кода, който ги използва. Това обаче е за сметка на допълнително усложняване на кода, което не е много желателно.

Характеристики:

- Създава семейства от свързани или зависими обекти без специфициране на конкретен клас.
- Използва дъщерни конкретни класове-фабрики за обекти, за да произвежда специализирани по тип обекти, имащи общ родител.
- Кодът, който ползва тези обекти не се интересува от спецификата им, а само от общия интерфейс.

Пример:

```
/*
 * GUIFactory example
 */

public abstract class GUIFactory {
    public static GUIFactory getFactory() {
        int sys = readFromConfigFile("OS_TYPE");
        if (sys == 0) {
            return(new WinFactory());
        } else {
            return(new OSXFactory());
        }
    }
    public abstract Button createButton();
}

class WinFactory extends GUIFactory {
    public Button createButton() {
        return(new WinButton());
    }
}

class OSXFactory extends GUIFactory {
    public Button createButton() {
        return(new OSXButton());
    }
}
```

```

}

public abstract class Button {
    private String caption;
    public abstract void paint();

    public String getCaption(){
        return caption;
    }
    public void setCaption(String caption){
        this.caption = caption;
    }
}

class WinButton extends Button {
    public void paint() {
        System.out.println("I'm a WinButton: " + getCaption());
    }
}

class OSXButton extends Button {
    public void paint() {
        System.out.println("I'm a OSXButton: " + getCaption());
    }
}

public class Application {
    public static void main(String[] args) {
        GUIFactory aFactory = GUIFactory.getFactory();
        Button aButton = aFactory.createButton();
        aButton.setCaption("Play");
        aButton.paint();
    }
    //output is
    //I'm a WinButton: Play
    //or
    //I'm a OSXButton: Play
}

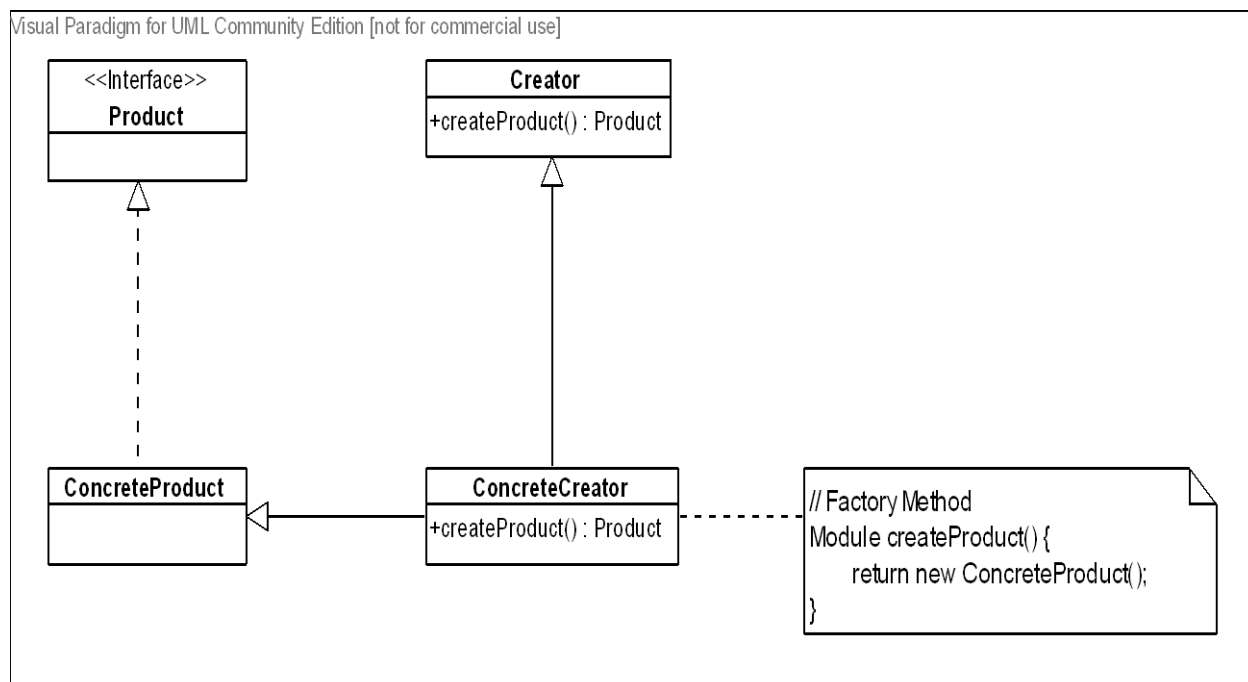
```

1.2. Метод Фабрика (*Factory Method*)

Метод Фабрика дефинира интерфейс за създаване на обекти, но точния тип на създадения обект се решава от наследниците на фабриката. Шаблонът Фабрика позволява инстанцирането на обекти по време на изпълнение на програмата. Наречен е така, защото е отговорен за "производството" на обектите. При параметризираният метод фабрика (*Parameterized Factory*), името на класа, който трябва да инстанцира се получава като аргумент.

Фабриката има за цел инстацирането на различни обекти, чиито типове не са предефинирани. Обектите се създават динамично в зависимост от параметрите, предадени на фабриката. По принцип фабриките са единствени в дадена програма,

затова за създаването им често се използва шаблонът Сингълтон.



Фиг. 1.2 - UML диаграма на Метод Фабрика

Най-често шаблонът „Метод Фабрика“ се използва когато:

- ◆ Искаме да предоставим точка на разширение за наследниците на даден клас при инстанциране на обект.
- ◆ Не искаме да използваме обикновен конструктор и искаме да енкапсулираме създаването на конкретен обект.
- ◆ Искаме да създаваме йерархии от обекти (като част от по-обхватния шаблон Абстрактна Фабрика.)
- ◆ Искаме да предоставим по-добро и по-смислено име на конструктора на даден клас.
- ◆ Искаме да контролираме броя на създадените обекти:
 - може да връща една и съща инстанция многократно;
 - да забавим създаването на даден обект до първото му поискване (*Lazy-loading*);
- ◆ Искаме да избегнем оператора `new`, който е опасен и може да има странични ефекти;
 - Понякога на обект не е сигурно, че ще бъде успешно – може да хвърли изключение, което не е добре да се случва в конструктор.
 - Използва се за създаване на обекти, зависещи от интернет услуги, файлова система, потоци или други обекти на операционната система.

◆ Много често се използва като „Шаблонен Метод“ (*Template Method*) за създаване на обекти.

- Използва се за създаване на сложни обекти, които имат нужда от повече конфигуриране.

Фабриката в обектно ориентираното програмиране е обект, който създава други обекти. Използва се като абстракция на конструктор, който може да бъде използван в много сценарии.

Участници

» Продукт (*Product*)

- Базов клас/интерфейс на обект, който ще бъде създаван от базовата фабрика.

» Конкретен продукт (*Concrete Product*)

- Конкретна имплементация или наследник на базовия клас продукт. Това е обектът, който ще бъде създаван от конкретната фабрика.

» Създател (*Creator*)

- Базовата имплементация на класа фабрика.

Декларира виртуален метод (*Factory Method*), който създава/върща продукт.

Методът може да бъде абстрактен в някои сценарии.

» Конкретен създател (*Concrete Creator*)

- Наследник на базовата Фабрика. Предефинира (*override*) базовия метод “създател”, като създава и връща конкретен продукт.

Взаимодействия

Всички продукти произведени от фабриката са от базовия тип *Product*. Създавателят (*Creator*) имплементира и енкапсулира цялата базова функционалност по създаването на един продукт, но оставя съответните конкретни създаатели (*Concrete Creator*) да предефинират типа на продуктите, които да създадат.

Оценка на шаблона

-1- Шаблонът „Метод Фабрика“ дава предимство пред простото използване на конструктори, енкапсулирайки създаването на конкретен обект. Много често се налага, когато се създаде един обект да се конфигурира по специален начин. Това конфигуриране заедно със самото създаване може много лесно да се скрие (енкапсулира) в един метод, вместо да бъде копирано навсякъде, където създаваме обекти.

-2- Използването на шаблона води до подобряване качеството на кода, защото се програмира срещу общ интерфейс на продуктите, а не конкретни техни

имплементации.

-3- Предоставя се точка на разширение за наследниците на даден клас при създаването на даден продукт. Наследниците на обекта създадел могат да предефинират метода фабрика и по-този начин да променят типа на връщания обект (конкретен продукт), запазвайки базовата функционалност. (Виртуален конструктор).

-4- Може да служи като средство за по-добра абстракция, когато работим с различни йерархии от класове.

-5- Един от основните недостатъци е, че Фабриките работят с йерархии от класове и ако продуктите не споделят общ интерфейс (базов клас), то не може да се използва шаблона „Метод Фабрика“.

Пример:

```
abstract class Pizza {
    public abstract double getPrice();
}

class HamAndMushroomPizza extends Pizza {
    private double price = 8.5;

    public double getPrice() {
        return price;
    }
}

class DeluxePizza extends Pizza {
    private double price = 10.5;

    public double getPrice() {
        return price;
    }
}

class HawaiianPizza extends Pizza {
    private double price = 11.5;

    public double getPrice() {
        return price;
    }
}

class PizzaFactory {
    public enum PizzaType {
        HamMushroom,
        Deluxe,
        Hawaiian
    }

    public static Pizza createPizza(PizzaType pizzaType) {
        switch (pizzaType) {
            case HamMushroom:
                return new HamAndMushroomPizza();
            case Deluxe:
                return new DeluxePizza();
        }
    }
}
```

```

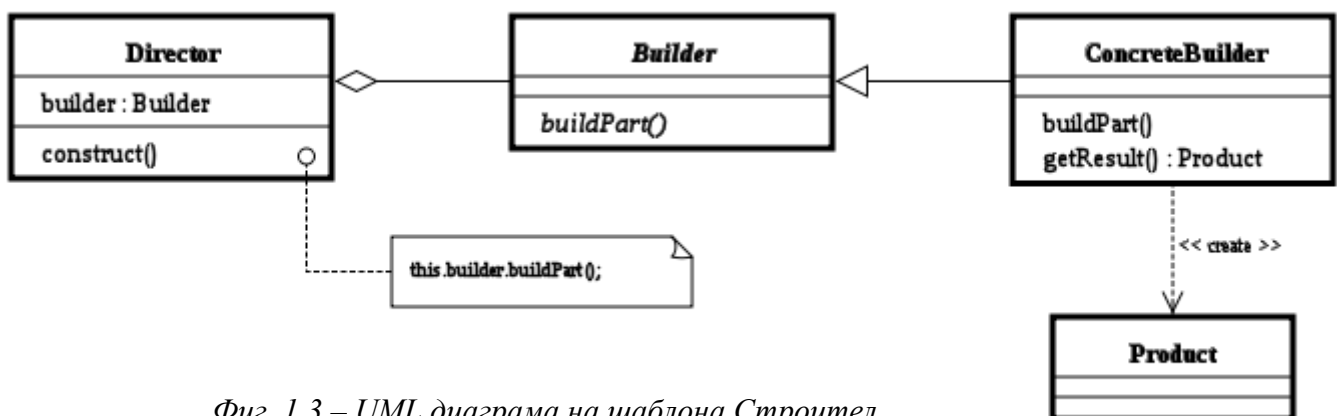
        case Hawaiian:
            return new HawaiianPizza();
    }
    throw new IllegalArgumentException("The pizza type " + pizzaType + " is
not recognized.");
}
}

class PizzaLover {
    /**
     * Create all available pizzas and print their prices
     */
    public static void main (String args[]) {
        for (PizzaFactory.PizzaType pizzaType : PizzaFactory.PizzaType.values()) {
            System.out.println("Price of " + pizzaType + " is " +
PizzaFactory.createPizza(pizzaType).getPrice());
        }
    }
}

```

1.3. Строител (Builder)

Целта е да се раздели създаването на сложен обект от неговото представяне (интерфейс), за да може с един и същ процес да се създават обекти с различно представяне. Шаблонът Метод Фабрика понякога се използва за първоначално инициализиране на продукта на Строителя. Също може да се използва за първоначалното създаване на самия Строител.



Фиг. 1.3 – UML диаграма на шаблона Строител

Пример:

```

/** "Product" */
class Pizza {
    private String dough = "";
    private String sauce = "";
    private String topping = "";

    public void setDough(String dough) { this.dough = dough; }
    public void setSauce(String sauce) { this.sauce = sauce; }
    public void setTopping(String topping) { this.topping = topping; }
}

```

```

/** "Abstract Builder" */
abstract class PizzaBuilder {
    protected Pizza pizza;

    public Pizza getPizza() { return pizza; }
    public void createNewPizzaProduct() { pizza = new Pizza(); }

    public abstract void buildDough();
    public abstract void buildSauce();
    public abstract void buildTopping();
}

/** "ConcreteBuilder" */
class HawaiianPizzaBuilder extends PizzaBuilder {
    public void buildDough() { pizza.setDough("cross"); }
    public void buildSauce() { pizza.setSauce("mild"); }
    public void buildTopping() { pizza.setTopping("ham+pineapple"); }
}

/** "ConcreteBuilder" */
class SpicyPizzaBuilder extends PizzaBuilder {
    public void buildDough() { pizza.setDough("pan baked"); }
    public void buildSauce() { pizza.setSauce("hot"); }
    public void buildTopping() { pizza.setTopping("pepperoni+salami"); }
}

/** "Director" */
class Waiter {
    private PizzaBuilder pizzaBuilder;

    public void setPizzaBuilder(PizzaBuilder pb) { pizzaBuilder = pb; }
    public Pizza getPizza() { return pizzaBuilder.getPizza(); }

    public void constructPizza() {
        pizzaBuilder.createNewPizzaProduct();
        pizzaBuilder.buildDough();
        pizzaBuilder.buildSauce();
        pizzaBuilder.buildTopping();
    }
}

/** Клиент поръчва пица. */
class BuilderExample {
    public static void main(String[] args) {
        Waiter waiter = new Waiter();
        PizzaBuilder hawaiian_pizzabuilder = new HawaiianPizzaBuilder();
        PizzaBuilder spicy_pizzabuilder = new SpicyPizzaBuilder();

        waiter.setPizzaBuilder( hawaiian_pizzabuilder );
        waiter.constructPizza();

        Pizza pizza = waiter.getPizza();
    }
}

```

1.4. Късна инициализация (*Lazy initialization*)

Целта на късната инициализация е да се отложи във времето създаването на обект, изчисляването на стойност или на някакъв друг отнемащ ресурси процес, до момента в който не ни е нужен за първи път.

Пример:

```
import java.util.*;

public class Fruit
{
    private static final Map<String,Fruit> types = new HashMap<String,Fruit>();
    private final String type;

    // using a private constructor to force use of the factory method.
    private Fruit(String type) {
        this.type = type;
    }

    /**
     * Lazy Factory method, gets the Fruit instance associated with a
     * certain type. Instantiates new ones as needed.
     * @param type Any string that describes a fruit type, e.g. "apple"
     * @return The Fruit instance associated with that type.
     */
    public static synchronized Fruit getFruit(String type) {
        if(!types.containsKey(type))
        {
            types.put(type, new Fruit(type)); // Lazy initialization
        }
        return types.get(type);
    }
}
```

1.5. Сингълтон (*Singleton*)

(Още Сек, Единак). Този шаблон се използва обикновено в моделирането на обекти, които трябва да бъдат глобално достъпни за обектите на приложението, или обекти, които се нуждаят от максимално късна инициализация за пестенето на ресурси от паметта. Използва се тогава, когато трябва да има точно една инстанция на даден клас. Най-често срещаният пример за това е връзка към база от данни. Реализирането на този шаблон позволява на програмиста да направи тази единствена инстанция лесно достъпна за много други обекти.

Шаблонът по-скоро ограничава, от колкото да насърчава създаването на класове.

Целите на Сингълтон шаблона са:

- Да контролира създаването на обекти, да гарантира създаването на една единствена инстанция, но да предоставя гъвкавост за създаването на повече обекти, ако ситуацията се промени.
- Да предостави глобална точка на достъп до него.

Съществуват множество случаи, в които създаденият обект е уникален. В реалния свят, всеки един от нас е уникален. Тоест ако виплатим подобни обекти в програма, то тяхното инстанциране трябва да е ограничено до една единствена инстанция на всеки клас и да притежават по една глобална точка на достъп до всеки клас.

Приложимост

Сингълтонът се използва в следните случаи:

- ◆ трябва да съществува една единствена инстанция от даден клас, която може да се достъпва от известно място за достъп.
- ◆ единствената инстанция трябва да бъде отворена за разширение чрез наследяване и потребителите трябва да са способни да използват въпросната инстанция без модификация на кода.

Singleton
uniqueInstance : Singleton
Singleton() getInstance() : Singleton

Фиг. 1.4 – Структура на Сингълтон.

Оценка на шаблона

Сингълтонът притежава следните особености:

- Контролиране на достъпа до единствената инстанция - тъй като Сингълтонът енкапсулира неговата единствена инстанция, той може да определя времето и начина на достъп до него.
- Подобрява производителността – в случай, че имаме обект без състояние, който се създава многократно, Сингълтонът премахва необходимостта от постоянното създаване и унищожаване на обекти. Естествено, възможно е Сингълтонът, в някои сценарии да не бъде най-доброто решение. Като алтернатива, могат да се използват статичните методи на класа.
- Ограничаване на претрупването с имена - представлява подобрене над глобалните променливи. Предпазва от претрупване на пространството от имена с глобални променливи, които съхраняват уникални инстанции.
- Позволява по-прецизното дефиниране на операции и инстанции на класове. Сингълтон класът е възможно да се наследява, следователно е възможно конфигурирането на програма, така че по време на изпълнението и да се инстанцира желаният Сингълтон клас.
- Позволява променлив брой инстанции - възможно е създаването на повече от една инстанция на Сингълтона, като контролирането на създаването и броя

инстанции се извършва в метода предоставящ достъп до инстанцията на Сингълтона.

- Гъвкавост - шаблонът ни предлага по-голяма свобода от статичните методи в клас и от статичните класове. В случай на нужда, предлага възможност за промяна на дизайна, за да бъде възможно инстанцирането на повече от един обект. Освен това, статичните член функции не са никога *virtual* - така класовете наследници не могат да ги препокрият. За да изберем между използването на статичен клас и Сингълтон, се ръководим от следните разсъждения: ако имаме набор от функции, които трябва да бъдат държани заедно, тогава избираме *static* клас. Всичко останало, което се нуждае от достъп до ресурси, е желателно да бъде имплементирано като Сингълтон.

Пример:

```
/**
 * The Singleton object is implementation of class that allows only
 * one instance.
 */
public class Singleton {
    // Reference to a sole instance.
    private static Singleton instance = null;
    // Атрибут на инстанцията.
    private int data = 0;

    /**
     * The construvtor must be private, so that no client can
     * istantiate the Singleton object.
     */
    private Singleton() {
    }
    /**
     * Gives a reference to one sole instance.
     */
    public static Singleton getInstance() {
        if (instance == null)
            instance = new Singleton();
        return instance;
    }

    public int getData() {
        return data;
    }

    public void setData(int data) {
        this.data = data;
    }
}
```

1.6. Обектен пул (Object Pool)

Обектният пул (басейн) представлява набор от инициализирани и готови за употреба обекти. Когато приложната програма има нужда от даден обект, той се взима от пулт. Когато обектът вече не е нужен, той не се унищожава, а се връща обратно в пулт.

- Singleton на стероиди :)
- Същият вид ограничен достъп, но до N обекта от даден тип, не само 1.
- Ограничение по валидност на обекта.
- Ползва се, когато създаването на въпросните обекти е “скъпо”.

Пример:

```
import java.io.Reader;
import java.io.IOException;

public class ReaderUtil {
    public ReaderUtil() {
    }

    /**
     * Dumps the contents of the {@link Reader} to a
     * String, closing the {@link Reader} when done.
     */
    public String readToString(Reader in) throws IOException {
        StringBuffer buf = new StringBuffer();
        try {
            for(int c = in.read(); c != -1; c = in.read()) {
                buf.append((char)c);
            }
            return buf.toString();
        } catch(IOException e) {
            throw e;
        } finally {
            try {
                in.close();
            } catch(Exception e) {
                // ignored
            }
        }
    }
}
```

1.7. Прототип (*Prototype*)

Създава обекти с помощта на обект-прототип. Новите обекти се създават чрез клониране на прототипа, вместо с използване на конструктор. Този шаблон е базиран на един от системните методи в обектно-ориентирането програмиране, а именно метода – *clone*.

Характеристики:

- Създаване само на един обект (*x*) от всеки клас (*ClassX*).
- Всяка следваща инстанция от този клас се създава не чрез извикване на *new ClassX*, а чрез клониране на вече създадения прототипен обект *x*.
- Заменя тотално употребата на *new*.
- Цели ООП езици са базирани на тази идея – *JavaScript*.
- Удобен патерн за имплементация на полиморфизъм в *strong-typing* езиците.

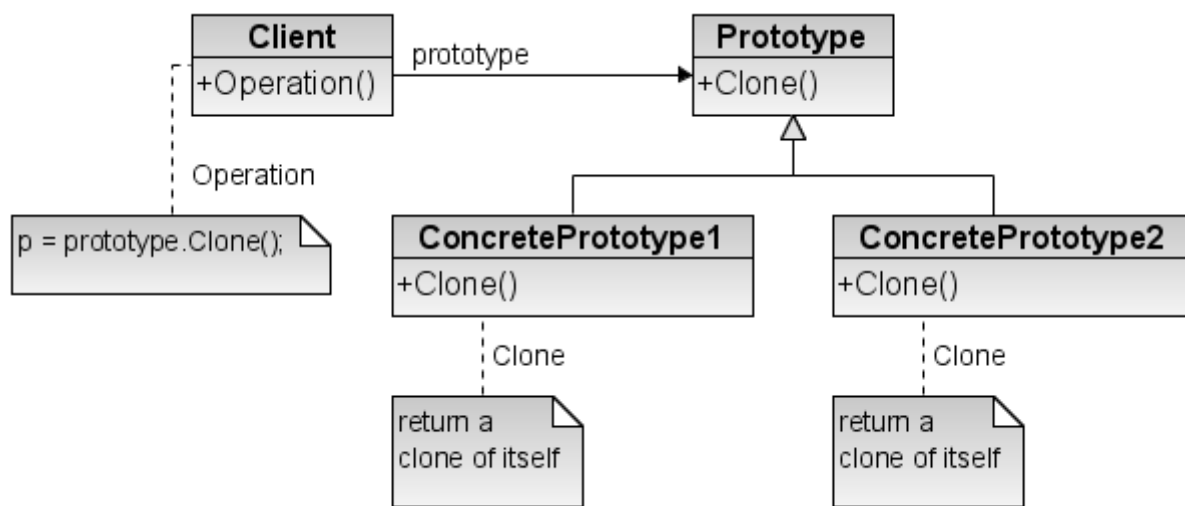
Приложимост

- ◆ Класовете за инстанциране се определят по време на изпълнение т.е. обектите се

създават, когато приложението вече е заредено в паметта, а не по време на стартирането на програмата.

◆ За избягване построяването на йерархия от класове на фабрики, които се създават по аналогия на йерархията от класове на продуктите.

◆ Инстанциите на един клас могат да имат само едно измежду няколко възможни състояния на класа т.е. когато създаваме обект, ние знаем, че неговото състояние ще е еднакво през цялото време. Пример за състояние на обект от тип бутон може да е дали той е активен (*enabled*) или неактивен (*disabled*).



Фиг. 1.5 – Схема на Прототип.

Оценка на шаблона

Шаблонът споделя голяма част от преимуществата на шаблоните Абстрактна фабрика и Строител. Тъй като всички тези шаблони описват йерархия от продукти, то общите преимущества са свързани точно с тази идея.

Останалите преимущества на шаблона се определят от другия участник – клиентът (*Client*), който осъществява ролята на мениджър на продуктите (*Product manager*).

Преимущества:

- ⊕ Добавяне и премахване на продукти по време на изпълнение (*run-time*) - шаблонът позволява добавянето и премахването на създадените продукти динамично. Единствено е необходимо да създадем прототипна инстанция, която използваме за нов обект. Тази инстанция се контролира и се намира в *Product manager* класа - клиентът в нашата система от участници.
- ⊕ Определяне на нови обекти с различни стойности на променливите - контролът върху създадените обекти обикновено е съсредоточен в *Product manager* класа. Този клас може да създава нови обекти като използва параметри в методите си при

генерирането на новити обекти.

- ⊕ Определяне на нови обекти с промяна структурата на класовете – създаване на нов конкретен прототип. Създаването на нови продукти е засегнато от връзките при наследяване в йерархията от продукти. Логично, когато те са променени и продуктите, които се генерират да са различни.
- ⊕ Намаляване наследяването между класове – съответно намаляване и броя на класовете. Считано за преимущество, използването на йерархия от продукти ще доведе до по-малко класове благодарение на точно това свойство.
- ⊕ Динамично конфигуриране на приложението с класове. *Product manager* класът може да осъществява динамичен контрол върху предлаганите продукти, т.е. от една страна както може да решава кога да бъдат създавани, той може да решава и кои прототипи са налични в момента.

Недостатъци:

- ◆ Всяка имплементация (*Concrete Prototype*) на интерфейса *Prototype* трябва да имплементира операцията по клониране. Понякога е възможно клониращата операция в конкретните продукти да бъде възпрепятствана поради едно или повече от следните условия:
 - при вече съществуващи класове може да е трудно предефинирането (*override*) на *clone()* метода;
 - проблем, когато някой вътрешен обект на прототипа не поддържа копиране;
 - проблем, когато прототипът има вътрешни обекти с циклични референци;
- ◆ Клониращата операция (сistemния метод *clone()*) е предпоставка за много грешки от програмистите.

Пример:

```
public class PrototypeManager {  
    public Car carPrototype = null;  
    public Truck truckPrototype = null;  
  
    public Car getCarPrototype(){  
        if(carPrototype == null)  
            initCarPrototype();  
  
        return carPrototype.clone();  
    }  
  
    public Truck getTruckPrototype(){  
        if(truckPrototype == null)  
            initTruckPrototype();  
    }  
}
```

```
        return truckPrototype.clone();  
    }  
}
```

Заклучение

Според мен актуалността на шаблоните за дизайн като цяло се дължи на концепцията, че няма смисъл да се открива наново решението на софтуерен проблем, след като то вече е било намерено и тествано успешно (т.е. да се открива топлата вода). Върху тази идея се гради софтуерното инженерство. По-специфично създаващите шаблони се използват за създаване на нови обекти, използвайки специален клас.

За търсенето на книгата “*Design Patterns*” дори в наши дни също има причини – както добрите обяснения и решения, така на изборът на език – C++, който е в основата на редица други обектно-ориентирани езици, запазил е голяма част от приложенията си, а разпространението на код под формата на хартиен носител не се ограничава до такава степен от правителствата, както цифровите оригинали.

Хубава новина в сферата е проектът за създаването на безплатна оригинална българска книга “Класически шаблони за дизайн”, чийто авторски колектив се ръководи от Светлин Наков, което само по себе си е гаранция за качество.

Използвана литература:

1. Addison Wesley - Gamma, Helm, Johnson, Vlissides – “Design Patterns, Elements of Reusable Object Oriented Software”, 1998
2. <http://design-patterns-book.googlecode.com/svn/trunk/chapters/02-work/>
3. http://sourcemaking.com/design_patterns
4. <http://www.dofactory.com/Patterns/Patterns.aspx/>
5. <http://www.oodesign.com/>